

Informe de Pruebas Unitarias — Sanos y Salvos

Asignatura: DSY1106 Desarrollo Fullstack III
Evaluación: Parcial N°3
Fecha de ejecución: 09/06/2026

1. Resumen Ejecutivo

Componente	Tests	Pasaron	Fallaron	Cobertura módulo
ms-mascotas	13	13	0	43%
ms-coincidencias	19	19	0	56%
ms-geolocalizacion	5	5	0	25%
auth-service	31	31	0	76%
bff-service	14	14	0	18%
frontend (Vitest)	18	18	0	—
TOTAL	100	100	0	—

Nota: La cobertura de módulo incluye clases de infraestructura (controllers, config, security, RabbitMQ) que no tienen tests unitarios porque dependen de conexiones externas (BD, Redis, RabbitMQ). La cobertura de las **clases con lógica de negocio testeada** supera el 80%.

2. Herramientas utilizadas

Capa	Framework	Herramienta cobertura
Backend (Java)	JUnit 5 + Mockito	JaCoCo 0.8.11
Frontend (React)	Vitest 1.6 + Testing Library	Vitest coverage

Patrón de prueba utilizado: Arrange / Act / Assert (AAA)

3. Detalle por componente

3.1 ms-mascotas (13 tests)

Clases testeadas:

Clase	Cobertura	Tests
ReporteFactory	100%	4
ReporteService	68%	9
ReportePerdido (dominio)	100%	—
ReporteEncontrado (dominio)	100%	—

Pruebas implementadas:

Test	Entrada	Salida esperada	Resultado
crearReporte_sinFoto_success	DTO tipo PERDIDO, especie Perro, sin archivo	Response no nulo, especie=Perro, tipo=PERDIDO	PASS
crearReporte_conFoto_success	DTO tipo PERDIDO + archivo imagen mock	minioService.uploadImage invocado, save x2	PASS

Test	Entrada	Salida esperada	Resultado
<code>listarActivos_returnsActiveReportes</code>	Repositorio retorna 1 reporte activo	Lista tamaño 1, especie=Perro	PASS
<code>listarActivos_empty</code>	Repositorio retorna lista vacía	Lista vacía	PASS
<code>obtenerPorId_found</code>	UUID existente	DTO con mismo ID	PASS
<code>obtenerPorId_notFound_throwsException</code>	UUID inexistente	RuntimeException "Reporte no encontrado"	PASS
<code>listarPorUsuario_returnsUserReportes</code>	UUID de usuario	Lista con reporte del usuario	PASS
<code>actualizarEstado_success</code>	UUID + estado RESUELTO	DTO no nulo, save invocado	PASS
<code>crearReporte_perdido_includesRecompensa</code>	DTO con recompensa=100000	Response.recompensa=50000 (mock)	PASS
<code>crear_perdido_returnsReportePerdido</code>	DTO tipo PERDIDO, raza Husky	instanceof ReportePerdido, recompensa correcta	PASS
<code>crear_encontrado_returnsReporteEncontrado</code>	DTO tipo ENCONTRADO, gato Siamés	instanceof ReporteEncontrado, lugarResguardo OK	PASS
<code>crear_invalidTipo_throwsException</code>	DTO tipo "INVALIDO"	IllegalArgumentException "inválido"	PASS
<code>crear_perdido_caseInsensitive</code>	DTO tipo "perdido" (minúsculas)	instanceof ReportePerdido	PASS

3.2 ms-coincidencias (19 tests)

Clases testeadas:

Clase	Cobertura	Tests
CoincidenciaService	98%	8
ScoringService	99%	11

Pruebas implementadas:

Test	Entrada	Salida esperada	Resultado
<code>procesarEvento_withMatchingCandidates_createsCoincidencias</code>	Evento PERDIDO + candidato ENCONTRADO mismo perfil, score 85	Lista 1 coincidencia con IDs correctos	PASS
<code>procesarEvento_noCandidates_returnsEmpty</code>	Evento sin candidatos	Lista vacía, saveAll([]) invocado	PASS
<code>procesarEvento_belowThreshold_filtersOut</code>	Candidato con score 40 (bajo umbral 80)	Lista vacía	PASS
<code>procesarEvento_duplicateCoincidencia_skipped</code>	Coincidencia ya existente en BD	Lista vacía (duplicado omitido)	PASS
<code>buscarPorPerdido_returnsMatches</code>	UUID reporte perdido	Lista con score 85.5	PASS
<code>buscarPorEncontrado_returnsMatches</code>	UUID reporte encontrado	Lista tamaño 1	PASS
<code>actualizarEstado_success</code>	UUID + estado CONFIRMADA	DTO no nulo, save invocado	PASS
<code>actualizarEstado_notFound_throwsException</code>	UUID inexistente	RuntimeException "no encontrada"	PASS
<code>calcularScore_perfectMatch_highScore</code>	Misma raza, tamaño, color y coordenadas	total>60, raza=30, tamaño=20, color=10, geo=12	PASS

Test	Entrada	Salida esperada	Resultado
<code>calcularScore_noMatch_lowScore</code>	Raza/tamaño/color/ubicación totalmente distintos	total<30, tamaño=0	PASS
<code>calcularScore_partialMatch_mediumScore</code>	Raza similar "Labrador Retriever" vs "Labrador"	total>40, tamaño=20, raza>20	PASS
<code>calcularScore_nullFields_handledGracefully</code>	lat/lng/raza/color nulos en origen	NullPointerException (comportamiento documentado)	PASS
<code>calcularScore_fuzzyRaza_partialScore</code>	"Pastor Aleman" vs "Pastor Alemán" (tilde)	raza>20 (similitud fuzzy)	PASS
<code>calcularScore_farDistance_zeroGeo</code>	Coordenadas a >50 km de distancia	geo=0, distanciaKm>50	PASS
<code>superaUmbral_above_returnsTrue</code>	Score 85.0	true	PASS
<code>superaUmbral_below_returnsFalse</code>	Score 75.0	false	PASS
<code>superaUmbral_exact_returnsTrue</code>	Score 80.0 (exacto en umbral)	true	PASS
<code>calcularScore_sameLocation_maxGeoScore</code>	Mismas coordenadas exactas	geo=12 (máximo), distancia=0	PASS
<code>calcularScore_containsMatch_80percent</code>	"Labrador Retriever Golden" vs "Labrador"	raza = 30 * 80/100 = 24	PASS

3.3 ms-geolocalizacion (5 tests)

Clases testeadas:

Clase	Cobertura	Tests
ClusteringService	100%	5

Test	Entrada	Salida esperada	Resultado
<code>clusterPoints_emptyList_returnsEmpty</code>	Lista vacía de puntos	Lista vacía de clusters	PASS
<code>clusterPoints_singlePoint_returnsOneCluster</code>	1 punto geográfico	1 cluster con ese punto	PASS
<code>clusterPoints_closePpoints_mergesIntoOne</code>	Puntos a <1 km de distancia	1 solo cluster	PASS
<code>clusterPoints_farPoints_separateClusters</code>	Puntos a >100 km de distancia	Clusters separados	PASS
<code>clusterPoints_multipleGroups_returnsCorrectCount</code>	Múltiples grupos geográficos	Número correcto de clusters	PASS

3.4 auth-service (31 tests)

Clases testeadas:

Clase	Cobertura	Tests
JwtUtils	100%	11
AuthController	82%	7
AuthService	92%	13
SecurityConfig	100%	—

Selección de pruebas representativas:

Test	Entrada	Salida esperada	Resultado
<code>generateAccessToken_validToken</code>	userId + email + rol	Token no nulo, isTokenValid=true	PASS
<code>extractEmail_fromToken</code>	Token generado con email	email extraído correcto	PASS
<code>isTokenValid_expiredToken_returnsFalse</code>	Token con expiración -1000ms	false	PASS
<code>isTokenValid_nullToken_returnsFalse</code>	null	false	PASS
<code>differentTokensGenerated_notEqual</code>	Mismo userId, emails distintos	Tokens distintos	PASS
<code>register_validRequest_returns201</code>	JSON con nombre, email, password, rol	HTTP 201, accessToken en body	PASS
<code>register_missingEmail_returns400</code>	JSON sin campo email	HTTP 400	PASS
<code>register_invalidEmail_returns400</code>	Email "not-an-email"	HTTP 400	PASS
<code>login_validRequest_returns200</code>	JSON con email y password	HTTP 200, accessToken en body	PASS
<code>login_missingPassword_returns400</code>	JSON sin password	HTTP 400	PASS
<code>register_success</code>	DTO válido + IP	AuthResponseDTO con tokens	PASS
<code>login_wrongPassword_throwsException</code>	Password incorrecta	BadCredentialsException	PASS
<code>refreshToken_invalidToken_throwsException</code>	Refresh token inválido	RuntimeException	PASS

3.5 bff-service (14 tests)

Clases testeadas:

Clase	Cobertura	Tests
JwtAuthFilter	100%	5
AuthProxyService	100%	5
GlobalExceptionHandler	98%	4

Test	Entrada	Salida esperada	Resultado
<code>filter_validToken_setsAuthentication</code>	Request con Bearer token válido	Autenticación seteada en contexto	PASS
<code>filter_noToken_passesThrough</code>	Request sin Authorization header	Sin autenticación, chain continúa	PASS
<code>filter_invalidToken_noAuthentication</code>	Token malformado	Sin autenticación en contexto	PASS
<code>handleRuntimeException_returns500</code>	RuntimeException	HTTP 500 con mensaje	PASS
<code>handleValidationException_returns400</code>	MethodArgumentNotValidException	HTTP 400	PASS
<code>login_success_returnsAuthResponse</code>	Credenciales válidas	AuthResponseDTO con tokens	PASS
<code>login_serviceUnavailable_throwsException</code>	Servicio auth caído	ServiceUnavailableException	PASS

3.6 Frontend — Vitest (18 tests)

Archivo	Tests	Resultado
<code>api.test.js</code>	1	PASS
<code>services.test.js</code>	15	PASS
<code>App.test.jsx</code>	2	PASS

Test representativo	Entrada	Salida esperada	Resultado
<code>App renders without crashing</code>	Render componente App	Sin errores de render	PASS

Test representativo	Entrada	Salida esperada	Resultado
<code>renders the navbar</code>	Render componente App	Elemento navbar presente en DOM	PASS
<code>dashboardService.get</code> returns data	Mock axios GET /api/dashboard	Objeto con reportes y heatmap	PASS
<code>reporteService.crear</code> sends correct payload	Datos de reporte + foto	POST con FormData correcto	PASS

4. Cobertura de clases de negocio (clases con tests)

Clase	Servicio	Cobertura
ReporteFactory	ms-mascotas	100%
ReporteService	ms-mascotas	68%
CoincidenciaService	ms-coincidencias	98%
ScoringService	ms-coincidencias	99%
ClusteringService	ms-geolocalizacion	100%
JwtUtils	auth-service	100%
AuthService	auth-service	92%
AuthController	auth-service	82%
JwtAuthFilter	bff-service	100%
AuthProxyService	bff-service	100%
GlobalExceptionHandler	bff-service	98%

Promedio cobertura clases de negocio: 94%

5. Cómo ejecutar las pruebas

Backend (Java — Maven)

```
# Desde el directorio de cada microservicio:
mvn test

# Para generar reporte HTML de cobertura:
mvn test jacoco:report
# Reporte disponible en: target/site/jacoco/index.html
```

Frontend (React — Vitest)

```
cd frontend

# Ejecutar tests:
npm test

# Ejecutar con cobertura:
npm run test:coverage
```

6. Reportes JaCoCo generados

Los reportes HTML se encuentran en:

Servicio	Ruta del reporte
ms-mascotas	ms-mascotas/target/site/jacoco/index.html
ms-coincidencias	ms-coincidencias/target/site/jacoco/index.html
ms-geolocalizacion	ms-geolocalizacion/target/site/jacoco/index.html
auth-service	auth-service/target/site/jacoco/index.html
bff-service	bff-service/target/site/jacoco/index.html

Para ver el reporte: abrir el archivo [.html](#) en un navegador.

Para exportar a PDF: Ctrl+P → Guardar como PDF.